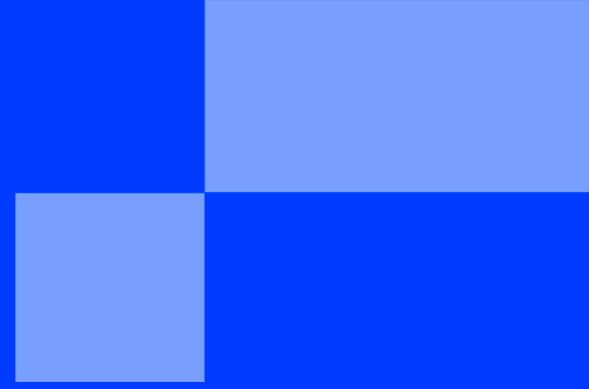
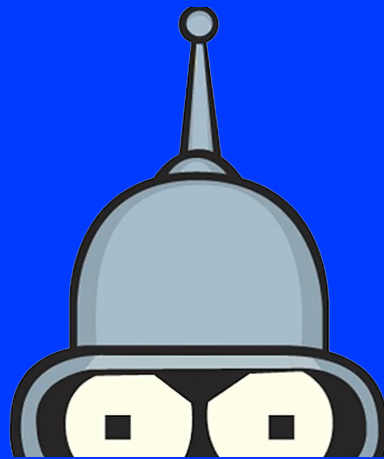




COREWAR - INSTRUCTIONS

< ELEMENTARY PROGRAMMING IN C />



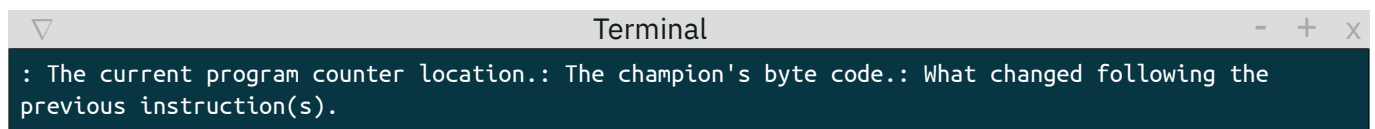
COREWAR - INSTRUCTIONS

The instructions

Champions have the ability of executing any of 16 instructions (detailed and established in the `op.c` source file provided). Below you will find a list explaining their parameters and behavior.

Don't worry if you don't yet understand everything written there; a lot of this information will actually come in handy during the Corewar project, not necessarily now!

In the dump examples given for the instructions below, the color code is as such :



```
Terminal
: The current program counter location.: The champion's byte code.: What changed following the
previous instruction(s).
```

For readability purposes, most of the examples below are shown as if there was only one champion running in the Corewar virtual machine.



Examples are not always illustrated in terminal boxes.
Terminal boxes do not always represent examples above them.

Take some time, read the byte code, and understand what the champion is doing. It is tedious, but necessary !

0x01 (live)

Parameters: direct `player_id`

Cycles: 10

Behavior:

Indicates that the player whose identifier equals `player_id` is alive.

0x02 (ld)

Parameters: direct/indirect `source`, register `destination`

Cycles: 5

Behavior:

Loads the value of the `source` into `destination`.

This operation sets the carry to 1 if `destination` is zero after the operation, otherwise 0.

Example:

`ld %34, r3` loads the number 34 into `r3`.

```
Terminal
Cycle: 0
Registers:
ld(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000007carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Terminal
Cycle: 5
Registers:
ld(1): alive
r1 : 00000001r2 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000007carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`ld 34, r3` loads `REG_SIZE` bytes starting at the address `PC + 34 % IDX_MOD` into `r3`.

0x03 (st)

Parameters: register `source`, indirect/register `destination`

Cycles: 5

Behavior:

Stores the value of `source` into `destination`.

Example:

`st r1, 32` stores the content of `r1` at the address `PC + 32 % IDX_MOD`.

```

Cycle: 0
Registers:
st(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00

```

```

Cycle: 5
Registers:
st(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000005carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00

```

```

Cycle: 10
Registers:
st(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000005carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00

```

st r1, r7 copies the content of r1 into r7.

0x04 (add)

Parameters: register `a`, register `b`, register `destination`

Cycles: 10

Behavior:

Adds the value of `a` to `b` and stores the result in `destination`

This operation sets the carry to 1 if `destination` is zero after the operation, otherwise 0.

Example:

`add r1, r2, r3` adds the content of `r1` and `r2` and puts the result into `r3`.

```
Terminal
Cycle: 5
Registers:
ld_add(1): alive
r1 : 00000001r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000007carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Terminal
Cycle: 15
Registers:
ld_add(1): alive
r1 : 00000001r2 : 0000002Ar4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 0000000Ccarry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

0x05 (sub)

Parameters: register **a**, register **b**, register **destination**

Cycles: 10

Behavior:

Subtract the value of **b** from **a** and stores the result in **destination**.

This operation sets the carry to 1 if **destination** would be negative, otherwise 0.

Example:

sub r1, r2, r3 subtract the content of r2 from r1 and puts the result into r3.

```
Terminal
Cycle: 5
Registers:
st_sub(1): alive
r1 : 00000001r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000007carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Terminal
Cycle: 15
Registers:
st_sub(1): alive
r1 : 00000001r2 : 0000002Ar4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 0000000C
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```


0x06 (and)

Parameters: direct/indirect/register `a`, direct/indirect/register `b`, register `destination`

Cycles: 6

Behavior:

Performs a binary AND between `a` and `b` and stores the result into the `destination`.

This operation sets the carry to 1 if `destination` is zero after the operation, otherwise 0.

Example:

and 1, %42, r2 puts `*(PC + 1) & 42` into `r2`.

```
Terminal
Cycle: 0
Registers:
and(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Terminal
Cycle: 6
Registers:
and(1): alive
r1 : 00000001r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000009carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

and r2, %0, r3 puts `r2 & 0` into `r3` and sets the carry to 1.

0x07 (or)

Parameters: direct/indirect/register [a](#), direct/indirect/register [b](#), register [destination](#)

Cycles: 6

Behavior:

Performs a binary OR between [a](#) and [b](#) and stores the result into the [destination](#).

This operation sets the carry to 1 if [destination](#) is zero after the operation, otherwise 0.

Example:

or 1, %42, r2 puts `*(PC + 1) | 42` into r2.

```
Terminal
Cycle: 0
Registers:
or(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Terminal
Cycle: 6
Registers:
or(1): alive
r1 : 00000001r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000009carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

or r2, %0, r3 puts `r2 | 0` into r3.

0x08 (xor)

Parameters: direct/indirect/register **a**, direct/indirect/register **b**, register **destination**

Cycles: 6

Behavior:

Performs a binary XOR between **a** and **b** and stores the result into the **destination**.

This operation sets the carry to 1 if **destination** is zero after the operation, otherwise 0.

Example:

xor 1, %42, r2 puts `*(PC + 1)` 42 into r2.

```
Terminal
Cycle: 0
Registers:
xor(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Terminal
Cycle: 6
Registers:
xor(1): alive
r1 : 00000001r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000009carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-----
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

xor r2, %0, r3 puts r2 0 into r3.

0x09 (zjmp)

Parameters: index `offset`

Cycles: 20

Behavior:

Perform a jump (move the PC) by `offset` bytes if the carry is worth 1. Otherwise, does nothing but consumes the same time.



This specific instruction **does not** have a coding byte !

It **always** takes a direct index (eg: a direct on `IND_SIZE` bytes).

Example:

`zjmp %25` jumps forward by 25 bytes ($PC = PC + 25$).

```

Cycle: 5
Registers:
ld_zjmp(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000007
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00

```

```

Cycle: 25
Registers:
ld_zjmp(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000020carry: 1
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00

```

0x0a (ldi)

Parameters: index/register `offset_a`, index/register `offset_b`, register `destination`

Cycles: 25

Behavior:

Adds together `offset_a` and `offset_b` (let's call that SUM) and loads the value at `PC + SUM % IDX_MOD` into `destination`.

This operation sets the carry to 1 if `destination` is zero after the operation, otherwise 0.

Example:

`ldi %1, %2, r3` reads `REG_SIZE` bytes from the address `PC + (1+2) % IDX_MOD` and copies them into `r3`.

```
Terminal
Cycle: 0
Registers:
ldi(1): alive
r1 : 00000000r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Terminal
Cycle: 25
Registers:
ldi(1): alive
r1 : 00000001r2 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000007carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`ldi 3, r4, r1` reads `IND_SIZE` bytes from the address `PC + 3 % IDX_MOD`, adds `r4` to this value. `REG_SIZE` bytes are read from the address `PC + SUM % IDX_MOD` and copied into `r1`.

0x0b (sti)

Parameters: register `source`, index/register `offset_a`, index/register `offset_b`

Cycles: 25

Behavior:

Adds together `offset_a` and `offset_b` (let's call that SUM) and stores the value of `source` into `PC + SUM % IDX_MOD`.

Example:

`sti r2, %4, %5` copies the content of `r2` into the address `PC + (4+5) % IDX_MOD`.

```
Terminal
Cycle: 0
Registers:
sti(1): alive
r1 : 00000000r2 : 00000001r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00
```

```
Terminal
Cycle: 25
Registers:
sti(1): alive
r1 : 00000000r2 : 00000001r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000007carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00
```

`sti r2, %:label, %5`, here we will call the value of `%:label label_offset`. It copies the content of `r2` into the address `PC + (label_offset+5) % IDX_MOD`, so 5 bytes after the `label`.

0x0c (fork)

Parameters: index `offset`

Cycles: 800

Behavior:

Creates a new program that inherits different states from the parent.

This program is executed at the address `PC + offset % IDX_MOD`.

Example:

`fork %32` will create a new program, starting at the address `PC + 32 % IDX_MOD`.

```
Terminal
Cycle: 0
Registers:
fork(1): alive
r1 : 00000000r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00
```

```
Terminal
Cycle: 800
Registers:
fork(1): alive
r1 : 00000000r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000003carry: 0

Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
```


0x0d (lld)

Parameters: direct/indirect `source`, register `destination`

Cycles: 10

Behavior:

Loads the value of the `source` into `destination`.

This operation sets the carry to 1 if `destination` is zero after the operation, otherwise 0.

Unlike the **ld** instruction, **lld** does not use the `IDX_MOD` modulo.

Example:

`lld 3072, r2` loads `REG_SIZE` bytes starting at the address `PC + 3072` into `r2`.

```

Cycle: 0
Registers:
lld(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
lld(2): alive
r1 : 00000002r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000C00carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
...
00000C00: 0D D0 0C 00 02 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00

```

```

Cycle: 10
Registers:
lld(1): alive
r1 : 00000001r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000005carry: 0
lld(2): alive
r1 : 00000002r2 : 0000DD0r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000C05carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
...
00000C00: 0D D0 0C 00 02 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00

```

0x0e (*lldi*)

Parameters: index/register `offset_a`, index/register `offset_b`, register `destination`

Cycles: 50

Behavior:

Adds together `offset_a` and `offset_b` (let's call that SUM) and loads the value at `PC + SUM` into `destination`. This operation sets the carry to 1 if `destination` is zero after the operation, otherwise 0.

Unlike the *ldi* instruction, *lldi* does not use the `IDX_MOD` modulo.

Example:

`lldi 2, %3072, r2` reads `IND_SIZ` bytes from the address `PC + 2`, adds 3072 to this value. `REG_SIZE` bytes are read from the address `PC + SUM` and copied into `r2`.

```

Cycle: 0
Registers:
lld(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
lld(2): alive
r1 : 00000002r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000C00carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
...
00000C00: 0E E4 00 02 0C 00 02 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00

```

```

Cycle: 50
Registers:
lld(1): alive
r1 : 00000001r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000005carry: 0
lld(2): alive
r1 : 00000002r2 : 00020C00r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000C05carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
...
00000C00: 0E E4 00 02 0C 00 02 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00

```

0x0f (lfork)

Parameters: index `offset`

Cycles: 1000

Behavior:

Creates a new program that inherits different states from the parent.
This program is executed at the address `PC + offset`.

Unlike the ***fork*** instruction, ***lfork*** does not use the `IDX_MOD` modulo.

Example:

`lfork %3072` will create a new program, starting at the address `PC + 3072 % IDX_MOD`.

0x10 (print)

Parameters: register `character`

Cycles: 2

Behavior:

Displays on the standard output the character whose ASCII code is the content of the register (in base 10). A 256 modulo is applied to this ASCII code.

Example:

`print r3` displays `*` if `r3` contains `42`.

```

Cycle: 0
Registers:
load_and_print_astek(1): alive
r1 : 00000001r2 : 00000000r3 : 00000000r4 : 00000000r5 : 00000000r6 : 00000000
r7 : 00000000r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000:
00000020:  00 00 00 00

```

```

Cycle: 30
Registers:
load_and_print_astek(1): alive

r8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000:
00000020:  00 00 00 00

```

```

Cycle: 42
Registers:
load_and_print_astek(1): alive
r1 : 00000001r2 : 00000041r3 : 00000073r4 : 00000074r5 : 00000065r6 : 0000006B
r7 : 0000000Ar8 : 00000000r9 : 00000000r10: 00000000r11: 00000000r12: 00000000
r13: 00000000r14: 00000000r15: 00000000r16: 00000000
PC : 00000000carry: 0
Memory:00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E
1F
-- -- -- -- --
00000000:
00000020:  00 00 00

```




Are you working on RobotFactory? There are things you might not understand : PC, IDX_MOD, ... Since these are the effects of instructions, they could be interesting elements for Corewar, but maybe not now. Still, you should keep it under your hat in the future, you never know.

v1

{EPITECH}